

ReBAC V2.3 — Compiler Intelligence

Branch: *ReBAC-V2.3* | **Builds on:** *V2.2 Dynamic Authorization DSL*

The Problem V2.2 Left Unsolved

The V2.2 compiler produces a correct RuleGroup IR, but it is **naïve** — it accepts redundant, contradictory, or dead rules without any feedback. A developer writing:

```
permission view =  
  doctor_of OR doctor_of
```

gets no warning. The redundant rule doubles the database lookups at runtime. Similarly:

```
permission view = true OR doctor_of
```

evaluates `doctor_of` needlessly — `true OR anything` is always `true`. These are compiler-detectable inefficiencies.

V2.3 adds **compiler intelligence**: static analysis, optimization passes, a linter, and an Explain Plan.

New Compiler Passes in V2.3

Pass 1: Type Checker

```
relation doctor_of : user  
relation contains : ward
```

Now the compiler enforces:

- `doctor_of` on a `Department` → ❌ Type Error at compile time
- `contains` on a `User` → ❌ Type Error at compile time

This mirrors Java's type system: `String x = 10;` fails at compile time, not runtime.

Symbol Table extension:

```
ResourceSymbol {
  relations: Map<name, { targetType: string }>
  permissions: Map<name, PermissionSymbol>
}
```

Pass 2: AST Optimizer

Three algebraic simplifications, proven correct by boolean algebra:

<code>NOT (NOT A)</code>	→ <code>A</code>	(double negation elimination)
<code>A OR A</code>	→ <code>A</code>	(idempotence)
<code>A OR true</code>	→ <code>true</code>	(constant folding)
<code>true AND A</code>	→ <code>A</code>	(identity element)

These are **sound rewrites** — they preserve the semantics of the original expression while reducing the size of the expression tree and the number of runtime evaluations.

Implementation: Single-pass AST transformer with structural pattern matching.

Pass 3: ADSL Linter

The linter operates on the **validated, typed AST** and emits `LintDiagnostic[]` :

Lint Code	Description	Example
UNUSED_RELATION	Declared but never referenced in any permission	<code>relation admin_of :</code> user but no permission uses it
SIMPLIFIABLE_EXPRESSION	Optimizer would simplify it	<code>doctor_of OR doctor_of</code> → redundant
CONSTANT_EXPRESSION	Always true or always false	<code>permission view = true</code>

Pass 4: Explain Plan

The Explain Plan generates a **human-readable execution tree** for a given (resource, permission) pair before runtime:

```
EXPLAIN patient.view
```

OR

```
├─ DirectLookup(doctor_of)      ← "Does user have doctor_of on
this patient?"
├─ DirectLookup(assigned_nurse) ← "Does user have assigned_nurse
on this patient?"
└─ RecursiveTraversal(contains → view)
    └─ ward.view
        ├─ DirectLookup(assigned_nurse)
        └─ RecursiveTraversal(contains → view)
            └─ department.view
                └─ DirectLookup(head_of)
```

This is the authorization equivalent of PostgreSQL's `EXPLAIN` command — it shows what the engine *will do* without actually running it.

Critical Review

✓ What V2.3 Gets Right

1. **The optimization passes are mathematically correct.** All three are provable boolean algebra identities.
2. **Linter design** is sound — it operates on post-type-check AST, meaning false positives are minimized.
3. **Explain Plan is a real operational necessity.** Authorization debugging is notoriously hard; Explain Plan is used by SpiceDB, OpenFGA, and Ory Keto for this exact reason.

⚠ Missing Optimization Passes

Missing: Dead branch elimination

```
doctor_of AND false → false    (always false – dead branch)
```

The entire AND subtree can be removed.

Missing: Common Subexpression Elimination (CSE)

```
(doctor_of AND contains.view)
OR
(admin AND contains.view)
```

`contains.view` is computed twice. CSE would cache the result of the first evaluation.

Missing: Topological reordering by cost The optimizer doesn't reorder branches by cost.

`contains.view` (30 DB lookups) before `doctor_of` (1 DB lookup) is wasteful. This is addressed in V3.3.

Missing: SSA Form (Static Single Assignment) Modern compilers convert to SSA form before optimization passes to enable more powerful analyses. ADSL's optimizer works directly on the AST tree without SSA.

✗ Incorrect: Explain Plan vs EXPLAIN ANALYZE

The V2.3 Explain Plan is **static** — it shows what *would* execute, not what actually executed.

PostgreSQL's `EXPLAIN ANALYZE` runs the query and shows actual timings. This is what V3.3's Execution Profiler provides.

The naming `Explain Plan` suggests the full `EXPLAIN ANALYZE` behavior but only delivers half of it in V2.3.

Comparison: Optimization in Real Compilers

Optimization	V2.3	GCC -O2	LLVM	Description
Constant folding	✓	✓	✓	<code>true OR x → true</code>
Dead code elimination	✗	✓	✓	Remove unreachable branches
CSE	✗	✓	✓	Cache repeated subexpressions
SSA form	✗	✓	✓	Enables most other optimizations
Inlining	✗	✓	✓	Inline computed usersets
Loop unrolling	N/A	✓	✓	Not applicable for trees

Summary

V2.3 elevates the ADSL compiler from a simple translator to an **intelligent compiler** with static analysis, algebraic optimization, and developer tooling. The type system, optimizer, and linter are all correctly implemented for an educational system. The main gaps — CSE, dead branch elimination, and the distinction between Explain Plan and EXPLAIN ANALYZE — are identified and will be addressed in V3.3.